

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / C Ruiz / D García / S villanova

Primer Semestre 2026

Universidad Andrés Bello

Departamento de Física y Astronomía



Resumen - Semana 6, Sesión 1 (Sesión 11)

Introducción y Repaso

Manipulación de Listas en Python

Introducción a NumPy

Ejercicios Prácticos

Conclusiones

Introducción y Repaso

- **En las sesiones previas** aprendimos:
 - Uso de bucles para calculos y creación de tablas
 - condicionales para toma de decisiones
 - Estructuras de control para análisis de datos
 - Funciones para encapsular operaciones complejas
- **Conexión con la sesión actual:**
 - Primero veremos listas de Python para trabajar colecciones de datos
 - Ahora exploraremos las limitaciones de estas estructuras nativas
 - Aprenderemos herramientas especializadas para computación científica

¿Por qué avanzar a NumPy?

Para abordar problemas físicos y astronómicos reales necesitamos herramientas computacionales más potentes y especializadas.

Introducción y Repaso

Objetivos de la Sesión 11

- **Comprender** los conceptos básicos de Listas en Python: creación, acceso, manipulación.
- **Comprender** los conceptos básicos de NumPy: arreglos (arrays), vectorización, broadcasting.
- **Practicar** la creación y manipulación de arrays en Google Colab.
- **Aplicar** estas herramientas a ejemplos numéricos y pequeños problemas de Física/Astronomía.

- **Rendimiento:** Las listas de Python no están optimizadas para cálculos numéricos masivos
- **Sintaxis verbose:** Operaciones matemáticas requieren bucles explícitos
 - Ejemplo: Multiplicar cada elemento de una lista por 2 → `[x*2 for x in lista]`
- **Funcionalidad limitada:** Sin soporte nativo para álgebra lineal, estadística avanzada
- **Heterogeneidad:** Las listas pueden contener diferentes tipos de datos, lo que impide optimizaciones

Problema en Ciencia y Física

Los cálculos científicos requieren alto rendimiento y operaciones optimizadas con grandes conjuntos de datos numéricos

- **Ejemplo:** Suma de todos los elementos en una secuencia de un millón de números
- Con **listas de Python**: Aproximadamente 10-100 veces más lento
- Con **NumPy**: Operaciones optimizadas a nivel de hardware

Importancia del Rendimiento

En problemas físicos reales (simulaciones de N-cuerpos, análisis de datos del CERN o telescopios), la diferencia de rendimiento puede significar horas o incluso días de tiempo de cómputo.

Hoy aprenderemos: Cómo aprovechar esta potencia para nuestros problemas científicos.

Manipulación de Listas en Python

Manipulación de Listas en Python

Creación y Acceso a Listas

- Las listas son **colecciones ordenadas y mutables**
- Creación y acceso:

```
1 # Creación de listas
2 numeros = [10, 20, 30, 40, 50]
3 nombres = ["Ana", "Carlos", "Elena"]
4 mixta = [1, "dos", 3.0, True]
5
6 # Acceso por índice (desde 0)
7 print(numeros[0])      # 10
8 print(numeros[-1])     # 50 (último elemento)
9
10 # Slicing [inicio:fin:paso]
11 print(numeros[1:4])    # [20, 30, 40]
12 print(numeros[:2])     # [10, 30, 50] (saltos de 2)
```

Manipulación de Listas en Python

Métodos para Agregar y Quitar Elementos

```
1 frutas = ["manzana", "naranja", "plátano"]
2
3 # Agregar elementos
4 frutas.append("uva")           # Al final
5 print(frutas) # ['manzana', 'naranja', 'plátano', 'uva']
6
7 frutas.insert(1, "pera")       # En posición específica
8 print(frutas) # ['manzana', 'pera', 'naranja', 'plátano', 'uva']
9
10 # Eliminar elementos
11 eliminado = frutas.pop()       # Elimina y devuelve el último
12 print("Eliminado:", eliminado) # Eliminado: uva
13 print(frutas) # ['manzana', 'pera', 'naranja', 'plátano']
14
15 frutas.remove("pera")          # Elimina por valor
16 print(frutas) # ['manzana', 'naranja', 'plátano']
17
18 del frutas[0]                  # Elimina por índice
19 print(frutas) # ['naranja', 'plátano']
```

Manipulación de Listas en Python

Unir y Separar Listas

```
1 # Unir listas
2 lista1 = [1, 2, 3]
3 lista2 = [4, 5, 6]
4
5 # Concatenación con +
6 combinada = lista1 + lista2
7 print(combinada) # [1, 2, 3, 4, 5, 6]
8
9 # Método extend
10 lista1.extend(lista2)
11 print(lista1)    # [1, 2, 3, 4, 5, 6]
12
13 # Multiplicar listas
14 repetida = [0] * 4
15 print(repetida)  # [0, 0, 0, 0]
16
17
```

Manipulación de Listas en Python

Métodos Adicionales para Listas

```
1  numeros = [3, 1, 4, 1, 5, 9, 2]
2
3  # Ordenar lista (modifica la original)
4  numeros.sort()
5  print(numeros)  # [1, 1, 2, 3, 4, 5, 9]
6
7  # Invertir lista
8  numeros.reverse()
9  print(numeros)  # [9, 5, 4, 3, 2, 1, 1]
10
11 # Contar ocurrencias
12 print(numeros.count(1))  # 2
13
14 # Encontrar índice (primera aparición)
15 print(numeros.index(5))  # 1
16
17 # Copiar una lista
18 lista_copia = numeros.copy()  # o list(numeros) o numeros[:]
19 print(lista_copia)  # [9, 5, 4, 3, 2, 1, 1]
20
21 # Verificar si existe un elemento
22 print(5 in numeros)  # True
23 print(7 in numeros)  # False
```

Manipulación de Listas en Python

Listas por Comprensión

- Forma concisa de crear listas basadas en listas existentes
- Estructura: `[expresión for item in iterable if condición]`

```
1 # Lista del cuadrado de números del 0 al 9
2 cuadrados = [x**2 for x in range(10)]
3 print(cuadrados) # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
4
5 # Lista de números pares del 0 al 9
6 pares = [x for x in range(10) if x % 2 == 0]
7 print(pares)    # [0, 2, 4, 6, 8]
8
```

Manipulación de Listas en Python

Caso Práctico con Listas: Análisis de Datos

```
1 # Temperaturas diarias de una semana (°C)
2 temperaturas = [22.5, 23.1, 21.8, 24.2, 25.0, 22.9, 21.5]
3
4 # Análisis básico manual
5 suma = sum(temperaturas)
6 prom = suma / len(temperaturas)
7 temp_max = max(temperaturas)
8 temp_min = min(temperaturas)
9
10 print(f"Promedio: {prom:.1f}°C")
11 print(f"Máxima: {temp_max}°C, Mínima: {temp_min}°C")
12 print(f"Rango: {temp_max - temp_min:.1f}°C")
13
14 # Identificar días con temperaturas > 23°C
15 dias_calurosos = [i+1 for i, t in enumerate(temperaturas) if t > 23]
16 print(f"Días calurosos: {dias_calurosos}") # [4, 5]
```

- Observa cómo podemos combinar `enumerate`, listas por comprensión e `if` para análisis.

Introducción a NumPy

Introducción a NumPy

¿Qué es NumPy?

- **NumPy** = *Numerical Python*
- **Biblioteca fundamental** para computación científica en Python
- Desarrollada para manejar eficientemente **grandes conjuntos de datos numéricos**
- Pilares principales:
 - **ndarray**: Estructura de datos multidimensional y homogénea
 - **Funciones vectorizadas**: Operaciones optimizadas sin bucles explícitos
 - **Herramientas matemáticas**: Álgebra lineal, transformaciones, estadística

Importancia en Física Computacional

NumPy es la base de prácticamente todo el ecosistema científico de Python: SciPy, Matplotlib, Pandas, scikit-learn y TensorFlow se construyen sobre NumPy

Instalación

`pip install numpy` o ya incluido en Google Colab, Anaconda

Introducción a NumPy

Creación de Arrays NumPy

```
1 import numpy as np
2
3 # Convertir lista de Python a array NumPy
4 lista = [1, 2, 3, 4]
5 arr = np.array(lista)
6 print("Array:", arr)
7 print("Tipo:", type(arr)) # <class 'numpy.ndarray'>
8
9 # Array de ceros
10 zeros = np.zeros(5)
11 print("Zeros:", zeros)
12
13 # Array de unos
14 unos = np.ones((2,3)) # 2 filas x 3 columnas
15 print("Unos:\n", unos)
16
17 # Rango de valores
18 rango = np.arange(0, 10, 2) # [0, 2, 4, 6, 8]
```

Introducción a NumPy

Atributos de un Array

```
1 arr = np.array([[1, 2, 3],
2                [4, 5, 6]])
3 print("Shape:", arr.shape) # (2, 3)
4 print("Dim:", arr.ndim)    # 2
5 print("Tipo de datos:", arr.dtype) # int64 (en sistemas Linux)
6 print("Total de elementos:", arr.size) # 6
```

- `shape` = tupla de dimensiones.
- `dtype` = tipo de dato (`int32`, `float64`, etc.).
- `ndim` = número de ejes.

Introducción a NumPy

Manipulación de Arrays y Operaciones

```
1 arr = np.array([10, 20, 30, 40, 50])
2 print(arr[0])      # 10
3 print(arr[1:3])    # [20 30]
4
5 mat = np.array([[1, 2, 3],
6                 [4, 5, 6],
7                 [7, 8, 9]])
8 # Acceder a fila 1, columna 2
9 print(mat[1, 2])   # 6
10
11 # Slicing de filas y columnas
12 submat = mat[0:2, 1:3] # Filas [0..1], Cols [1..2]
13 print(submat)
```

- **Ojo:** en NumPy, slicing crea **vistas**, no copias (con implicaciones en la modificación de datos).

Introducción a NumPy

Operaciones Elemento a Elemento (Vectorización)

```
1 x = np.array([1, 2, 3])
2 y = np.array([4, 5, 6])
3
4 print("Suma:", x + y)    # [5 7 9]
5 print("Producto:", x * y) # [4 10 18]
6 print("Potencia:", x**2)  # [1 4 9]
```

- Sin bucles explícitos.
- El núcleo de NumPy optimiza estas operaciones.

Ejemplo de Física: Ley de Gravitación Universal

```
1 # Calcular fuerza gravitacional entre masas
2 G = 6.674e-11 # Constante gravitacional en  $\text{N}\cdot\text{m}^2/\text{kg}^2$ 
3 m1 = np.array([1e5, 2e5, 3e5]) # masas en kg
4 m2 = 5e5 # masa en kg
5 r = np.array([10, 20, 30]) # distancias en m
6 F = G * m1 * m2 / r**2 # Ley de Newton, vectorizada
7 print("Fuerzas (N):", F)
```

Introducción a NumPy

Broadcasting

- Mecanismo que NumPy utiliza para manejar operaciones entre arrays de diferentes formas.
- Ejemplo básico:

```
1 a = np.array([1, 2, 3])
2 b = 2
3 # b se "expande" para coincidir con la forma de a
4 res = a * b # [2 4 6]
```

- Ejemplo en Física: Conversión de temperaturas

```
1 # Temperaturas en °C para varias ciudades durante una semana
2 temps_C = np.array([[25, 24, 26, 28, 27], # Ciudad 1
3                     [20, 19, 21, 22, 20], # Ciudad 2
4                     [30, 31, 32, 30, 29]]) # Ciudad 3
5
6 # Conversión a Fahrenheit:  $F = C \cdot 9/5 + 32$ 
7 temps_F = temps_C * 9/5 + 32
8
9 print("Temperaturas en °F:\n", temps_F)
```

Introducción a NumPy

Ejemplo: Sumar Fila a una Matriz

```
1 mat = np.array([[1,2,3],
2                 [4,5,6],
3                 [7,8,9]])
4 fila = np.array([10, 20, 30])
5
6 # Broadcasting: se suma fila a cada fila de la matriz
7 res = mat + fila
8 print(res)
9 # [[11 22 33]
10 #  [14 25 36]
11 #  [17 28 39]]
```

Aplicación en Astronomía

```
1 # Ejemplo: Calibrar lecturas de un detector astronómico
2 # Supongamos un detector 3x3 con valores brutos
3 lecturas = np.array([[100, 120, 115],
4                      [95, 105, 110],
5                      [130, 125, 135]])
6
7 # Factor de calibración para cada columna
8 calibracion = np.array([1.05, 0.98, 1.02])
9
10 # Aplicar calibración (broadcasting)
11 lecturas_calibradas = lecturas * calibracion
12 print("Lecturas calibradas:\n", lecturas_calibradas)
```

Introducción a NumPy

Funciones NumPy Comunes

```
1 arr = np.array([2, 4, 6, 8])
2
3 print("Suma total:", np.sum(arr))          # 20
4 print("Mínimo:", np.min(arr))             # 2
5 print("Máximo:", np.max(arr))            # 8
6 print("Promedio:", np.mean(arr))         # 5.0
7 print("Desv. Estándar:", np.std(arr))     # ~2.236
```

- También hay `np.median(arr)`, `np.var(arr)` y muchas más.
- Para arreglos 2D, se puede especificar `axis=0` o `axis=1`.

Ejemplo en Física: Análisis Estadístico

```
1 # Mediciones de velocidad de partículas (m/s)
2 velocidades = np.array([
3     [12.3, 13.1, 12.8, 12.5], # Experimento 1
4     [11.9, 12.7, 13.2, 12.9], # Experimento 2
5     [12.5, 13.0, 12.6, 12.3]  # Experimento 3
6 ])
7
8 print("Velocidad media por experimento:", np.mean(velocidades, axis=1))
9 print("Desviación estándar total:", np.std(velocidades))
```

Ejercicios Prácticos



Enunciado

- Crear un arreglo 1D con los números del 1 al 10.
- Imprimir su **media**, **suma** y **producto** de todos los elementos.
- Reemplazar los valores ≤ 5 por 0, y los restantes por 1 (opcional: slicing lógico).
- Mostrar el arreglo resultante.

Conceptos: Arrays, operaciones vectorizadas, slicing lógico.

Física relevante: Manipulación de datos experimentales.

Sugerencia: Usa `np.arange` y operaciones de comparación (`arr <= 5`).



Enunciado

- Crear una matriz 3x3 con $\{[1, 2, 3], [4, 5, 6], [7, 8, 9]\}$.
- Definir un **vector** $[10, 20, 30]$.
- Sumar dicho vector a cada fila de la matriz (broadcasting).
- Mostrar el resultado.

Conceptos: Manipulación 2D, broadcasting, suma de vectores.

Física relevante: Suma de señales o datos experimentales.



Enunciado

- Usar `np.arange` para generar un rango de n valores.
- Calcular una aproximación de π usando la serie de Leibniz:

$$\pi \approx 4 \sum_{k=0}^n \frac{(-1)^k}{2k+1}$$

- Comparar el valor obtenido para distintos n .

Conceptos: Vectorización, sumas numéricas, aproximación de constantes.

Física relevante: Métodos numéricos en física y matemáticas.

Sugerencia: Emplear vectorización para computar $\frac{(-1)^k}{2k+1}$.



```
1 # Creamos el arreglo y calculamos estadísticas
2 import numpy as np
3 arr = np.arange(1, 11) # [1..10]
4 print("Array:", arr)
5 print("Media:", np.mean(arr)) # 5.5
6 print("Suma:", np.sum(arr)) # 55
7 print("Producto:", np.prod(arr))# 3628800
8
9 # Reemplazo valores <=5 con 0, y >5 con 1
10 arr_mod = arr.copy()
11 arr_mod[arr_mod <= 5] = 0
12 arr_mod[arr_mod > 5] = 1
13 print("Array modificado:", arr_mod)
```

Discusión: El uso de slicing lógico permite modificar el arreglo de forma eficiente y sin bucles.



```
1 # Suma de vector a matriz usando broadcasting
2 import numpy as np
3 mat = np.array([[1,2,3],
4                 [4,5,6],
5                 [7,8,9]])
6 fila = np.array([10, 20, 30])
7 resultado = mat + fila # Broadcasting
8 print("Matriz resultante:\n", resultado)
9 # [[11 22 33]
10 #  [14 25 36]
11 #  [17 28 39]]
```

Discusión: Broadcasting permite sumar el vector a cada fila sin bucles explícitos.



```
1 # Aproximación de pi usando la serie de Leibniz
2 import numpy as np
3
4 n = 1000000 # término superior
5 k = np.arange(n+1) # [0..n]
6
7 # (-1)^k = np.power(-1, k)
8 # denominador = (2*k + 1)
9 terminos = ((-1)**k) / (2*k + 1)
10 aprox_pi = 4 * np.sum(terminos)
11 print("Aproximación de pi con n =", n, ":", aprox_pi)
12
13 # Comparar con valor real
14 import math
15 print("Diferencia:", abs(math.pi - aprox_pi))
```

Discusión: La vectorización permite calcular sumas grandes de manera eficiente y rápida.

Ejercicios Prácticos

Discusión de Resultados

- **Verificación:** ¿Coincidió con tus resultados o surgieron discrepancias?
- **Rendimiento:** usar vectorización en lugar de bucles manuales.
- **Optimización:** manipulación y slicing evitan copias innecesarias.

Conclusiones

- **Aprendimos** los conceptos fundamentales de NumPy y su utilidad para cálculos numéricos.
- **Practicamos** la creación, manipulación y operaciones vectorizadas con arrays.
- **Exploramos** aplicaciones en problemas matemáticos y de simulación.
- **Desarrollamos** habilidades de análisis numérico y trabajo colaborativo.

Habilidad adquirida

Uso eficiente de arrays y operaciones vectorizadas para resolver problemas científicos.

- **Sesión siguiente:** Operaciones avanzadas con NumPy (reshaping, álgebra lineal) y visualización con Matplotlib.
- **Recomendación de estudio:** Repasa la documentación oficial de NumPy y experimenta con ejemplos propios.
- **Práctica sugerida:** Resuelve ejercicios adicionales con datos reales o simulados.

¡Nos vemos en la próxima sesión!